Where to Stop:
Tile-Adaptive Early Exit in a Learned Image Decoder

Anonymous CVPR submission · Paper ID ****

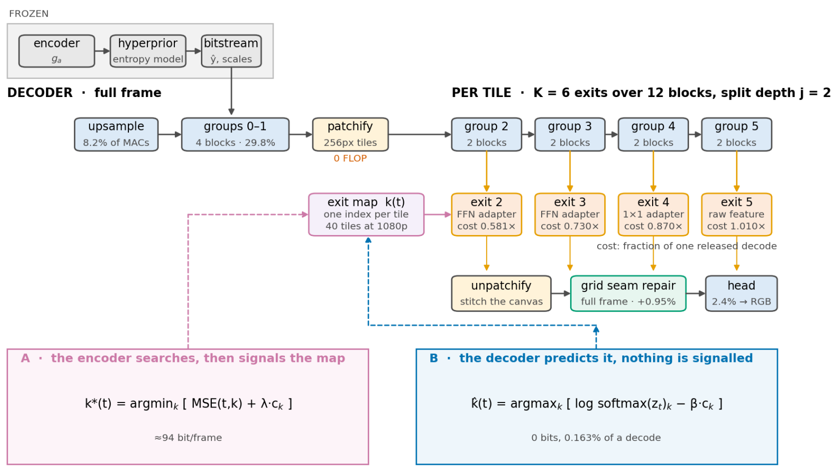

Figure 1. FLEX-UF end to end. Grey is frozen and never touched; blue is inherited from DCVC-UF and fine-tuned; orange and green are new. The first j block groups run over the whole frame and the rest run per tile on a shrinking active set. The exit map that drives the shrinkage comes either from an encoder-side search (A) or from a decoder-side predictor (B).

Abstract

A learned image decoder runs the same graph on every frame whatever the frame contains. We add an early-exit ladder to the intra decoder of DCVC-UF, cut each frame into tiles, and let every tile leave the trunk at whichever of K exits its content needs. The encoder and the entropy model stay frozen and the coded payload is unchanged, so the method applies to bitstreams that already exist. On the 53-sequence common test set a 0.1 dB budget removes 29.8% of the decoder's multiply-accumulates at the lowest rate and 15.6% at the highest, for a BD-Rate cost of 0.88%.

We also characterise when such a budget does anything at all. Below a floor set by tiling no allocation is feasible; above a saturation point every tile already sits on the cheapest rung; and measuring the budget in units of that window collapses five rate curves, 15.5 points apart at a matched decibel, to within 1.7. Tiling dominates the cost and scales with per-tile depth rather than with contaminated area. Finally, a rule with no parameters that routes on the bits the entropy model has already spent per tile beats our trained 144 K-parameter router at every rate by up to 3.4 points, which we report as a control that work on adaptive inference ought to include.

1. Introduction

Learned codecs are compared on rate and distortion, with complexity given as a single number, so many GMAC per frame or so many milliseconds. That number is a constant. A learned decoder runs the same graph on a page of text as on a cloudless sky, although the sky is much the easier picture.

Classification networks abandoned this a decade ago. Early-exit architectures attach classifiers at intermediate depths and stop as soon as the prediction is confident [3, 15, 20], and the literature around them is by now mature. Super-resolution adopted the same idea spatially. ClassSR [12] routes image patches to networks of different capacity by difficulty, and APE [1] exits patches at different depths of one network. Learned compression has taken a different route. Slimmable autoencoders [22, 23] give one model several complexity levels, but the level is chosen *per stream* rather than per region, and switching it changes the bitstream.

We ask the spatial-adaptivity question inside a learned decoder, under a constraint that makes the answer deployable: **the encoder is frozen and the coded payload is unchanged.** Whatever we do has to consume the bitstream the released encoder already produces. That rules out re-training the analysis transform, changing the entropy model, or altering the latent. It leaves one place to spend adaptivity, the synthesis transform.

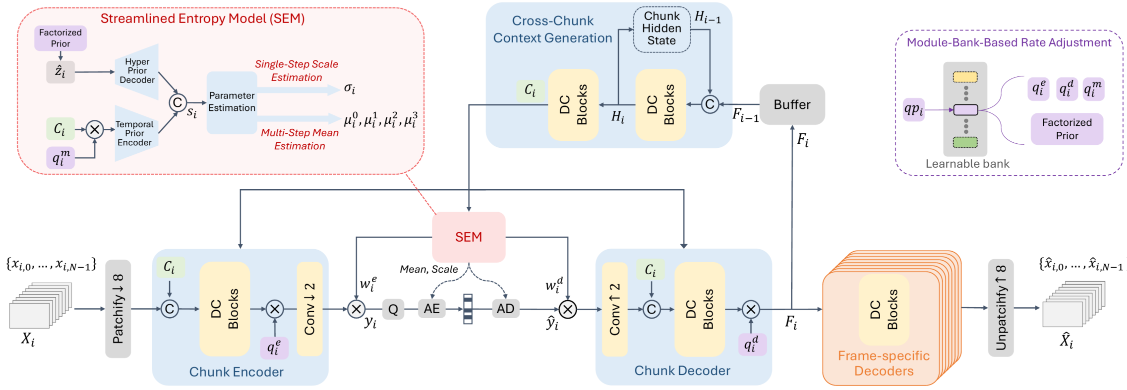

Figure 2. The decoder we modify. Figure 3 of DCVC-UF [14], reproduced. Everything up to the reconstruction stays frozen in this work: the patch embedding, the chunk encoder, the entropy model and the coded payload. FLEX-UF replaces the frame-specific decoders on the right with a ladder of exits taken per tile.

Our method, FLEX-UF, is an exit ladder over the twelve residual blocks of the DCVC-UF intra decoder. The first j blocks run over the whole frame. The rest run per tile, over a set of tiles that shrinks with depth, and the shrinkage is where the saving comes from. Each exit hands its feature to the shared head through a small pointwise adapter, zero-initialised so that at step zero the deepest exit reproduces the released decoder bit-exactly.

Getting this to work depended less on the ladder than on two problems that have little to do with early exit as it is usually studied.

Tiling has a large price.

Spatial adaptivity needs tiles, and once a frame has been cut into tiles, every 3×3 convolution at a tile border reads invented values. Under the stock padding rule the resulting error on this decoder is 0.677 dB at high rate. That is several times the entire budget the method works to, and it is paid before any tile has saved a single operation. In Section 4 we treat padding as an *estimator* of the unseen neighbour and measure four of them. The ordering we get is not the obvious one.

A quality budget has a working range.

Tiling costs something even when nothing exits early, so there is a *floor*, and budgets below it admit no allocation at all. The ladder also has a shallowest rung, so there is a *saturation* point above which every tile already sits on that rung and more quality buys nothing. Between the two the budget genuinely trades quality for compute. We measure both ends at every rate in Section 5.4, where the working budget uses 45% of that window at low rate and 9% at high rate.

Contributions.

(i) A tile-adaptive early-exit ladder for a learned image decoder, deployable against existing bitstreams because the encoder and the coded payload are untouched. It saves 29.8% to 15.6% of decoder MACs for 0.1 dB on the full CTC set, at a BD-Rate cost of 0.88%.

(ii) The operating structure of a quality budget: a floor below which no allocation is feasible, a saturation point above which none improves, both in closed form, and seven propositions verified numerically rather than asserted. Measuring the budget in units of that window accounts for most of the rate dependence, on two independently trained checkpoints.

(iii) A parameter-free control that adaptive-inference work is rarely measured against. Routing on the bits the entropy model has already spent per tile costs nothing, adds nothing to the stream, and beats our trained head at every rate, while agreeing with the oracle on fewer tiles than the head does.

2. Related work

Early exit.

BranchyNet [20] and MSDNet [3] established the pattern of intermediate classifiers with a confidence rule; SDN [15] framed it as mitigating overthinking. We train all exits jointly, after Scardapane et al. [18], and distil between exits as in Phuong and Lampert [17]. The caveat in [19] is that too large a student-teacher gap hurts the shallowest exits, so our distillation runs between *adjacent* exits. All of this work exits on a confidence signal computed from the network's own output. A decoder has no such signal. There is no class posterior, and the quantity that would decide is the error against a source the decoder cannot see (Section 3.4).

Spatially adaptive inference.

ClassSR [12] sorts super-resolution patches into easy/medium/hard and runs a different network on each; APE [1] exits patches at different depths; Glance-and-Focus [8] spends resolution adaptively. We share the per-region premise with all three, and we inherit the same tiling problem, though the cost of tile borders is rarely quantified in that line of work. To our knowledge the estimator view of border padding and its measured ordering (Section 4) is new. The closest prior work is the per-channel AR(1) padding of Kaseva et al. [11], which we implement and evaluate.

Method	What varies	Granularity	New bitstream
SlimCAE	width	per stream	yes
Slimmable video	width	per stream	yes
EVC	mask / pruning	per model	yes
Spatial competition	which codec	per region	yes
DCVC-RT	architecture	fixed	yes
FLEX-UF	**decoder depth**	**per region**	**no**

Table 1. Where this sits. Complexity control in learned compression varies the model; we vary how much of a fixed model runs where. The last column is what makes the difference operational. Every other row requires a decoder that matches the encoder that produced the stream.

Complexity control in learned compression.

SlimCAE [22] and slimmable video codecs [23] expose several widths of one model. The choice is per stream and it changes the encoder, so the bitstream is not interchangeable. DCVC-FM [13] and DCVC-UF [14] reduce cost architecturally, for every frame equally. Rate–distortion–complexity has since become an explicit third axis. Gao et al. [35] tune spatial context usage to trade decode cost against rate, Ho et al. [36] survey where conditional residual coding sits on that surface, and Zhang and Gao [37] route *whole frames* to one of several jointly-trained coding paths, spending less computation on cheaper frames. Every one of these changes the model, and with it what the encoder emits. Our axis is orthogonal. The model and the bitstream are both fixed, and only *how much of the decoder runs where* varies.

The closest neighbour.

Blard et al. [27] also partition an image into regions, also choose per region by a rate–distortion cost computed at the encoder, and also transmit a mode map, which is the skeleton of our configuration A. The differences are in what the regions choose among and in what the map buys. Their regions choose among several *separately trained* codecs, so the decoder must hold all of them and its complexity is that of one codec regardless of the choice; the map buys rate, not compute. Ours choose a prefix length of a single trunk, so the weights are shared by construction and the map buys compute at fixed rate. Because their alternatives are unrelated networks, no decoder-side predictor could stand in for the encoder's search. The nesting that makes our exits prefixes of one another is exactly what makes configuration B possible at all.

Signalling versus prediction.

Being *told* a mode decision rather than inferring it is the norm in standardised video coding. HEVC [9] and VVC [21] transmit partitioning, prediction mode and transform tree. We evaluate both and treat the signalled variant as the conventional design.

Deferring to an oracle under a budget.

Configuration C, where a predictor decides most cases and a small budget of the hardest ones is handed to something exact, is the shape of selective prediction [38] and learning to defer [39, 40], and of the budgeted variant of the latter [41]. Two things differ, and both make our case easier. The expert here is the encoder's own search, so it is exact and always available at no test-time cost. What is scarce is the *bits* needed to say what it decided. The selection rule is not learned either. Because the objective is separable over tiles, the optimal set of size s at a fixed multiplier is exactly the s largest regrets, which the encoder can compute. Who to defer and how to learn it is the open question in that literature, and it has a closed form here. What remains is the question we measure, which is how concentrated the regret is.

Allocating a budget over units.

The construction we use is not new and we do not present it as such. Shoham and Gersho [28] showed that for a finite set of per-unit operating points, a Lagrangian sweep decouples the allocation across units and traces exactly the lower convex hull of the achievable set; Ortega and Ramchandran [29] made it standard practice in image and video coding. What we add is the structure this particular operating set has: a floor below which no allocation is feasible, a saturation point above which none improves, and a measurement of what the convex-hull restriction costs (Section 5.4). The same relaxation has resurfaced for test-time compute in language models [30], with per-instance decoupling and a binary search on the multiplier. That is the identical structure in a domain with no rate axis, and we read it as evidence that the floor/saturation characterisation is worth stating generally.

How much is there to gain?

Bounding what adaptive inference could achieve is itself a line of work. Hasan et al. [34] derive an oracle bound on efficiency at fixed accuracy, given per-model resource and accuracy, and report 43–121× on ImageNet and 7–81× on HellaSwag. Their bound has a ceiling and no floor, because the largest model in their family reaches the reference accuracy by definition. Spatial adaptivity introduces a floor. Cutting a frame into tiles costs quality even when every tile runs to full depth, so the reference is unreachable at *any* compute and the feasible set of budgets is an interval rather than a ray. Section 5.4 characterises that interval and both of its ends.

Tile boundaries.

Every method that processes an image in independently-computed tiles meets the same artefact. The remedies in the literature are overlap and averaging, local padding from neighbouring patches [31], training with overlaps [32], and fitted extrapolation [11]. Local padding is the closest to the exact remedy we describe in Section 4.4, and the difference is accounting. It pads every convolutional layer and does not report the cost. We pad only the 0.29% of each block that has any spatial extent, which is what makes the exact fix cost 0.032% of the decode rather than a multiplier on all of it. To our knowledge the estimator view of the padding rule, the measured ordering of four estimators, and the dependence of the penalty on per-tile depth (Section 4) have not been reported.

Where the time goes.

DCVC-RT [33] argues that operational rather than computational complexity is the speed bottleneck for neural codecs, and cites channel reductions that yield linear rather than quadratic speedups. Section 5.9 is an instance of that claim inside one loop. Removing 35.3% of the operations buys 29.1% of the time, and 1.2 points of the difference come back from reordering the loop with the arithmetic untouched.

3. Method

3.1 Where the computation is

The DCVC-UF intra decoder is one upsampling block, twelve DepthConvBlocks and a head, costing 453.5 GMAC per 1080p frame. The twelve blocks are 89.4% of that, which is why we build the ladder across them. Inside one block at $C=384$ channels, the only operator with any spatial extent is a 3×3 depthwise convolution, and it costs $9C$ against the block's $8C^2+9C$. That is **0.29%.** We come back to the fraction several times below. Tile borders damage the picture through it, and it is what makes the exact remedy of Section 4.4 affordable at all. It is also why we kept the adapters pointwise.

3.2 The exit ladder

With K exits over the twelve blocks and split depth j, groups 0..j−1 run full frame for every tile and groups j..K−1 run per tile. A tile assigned exit k runs groups j..k and then leaves. Writing c_k for the cost of exit k in units of one released decode, the cost of a frame with exit map k is the mean of c over its tiles.

Two consequences follow, and both are easy to state wrongly. Exits shallower than j are indistinguishable from each other, because the first j groups run for every tile regardless, and the usable ladder therefore has K−j distinct costs. The second consequence is the *architectural ceiling* $S_{max} = 100(1-c_j)\%$, reached when every tile takes exit j. For our shipped setting (K=6, j=2, 256 px tiles) $S_{max} = 39.1\%$. We show in Section 5.4 that this bound is *reached* in practice at low rate, so it behaves as an operating point and not as an asymptote.

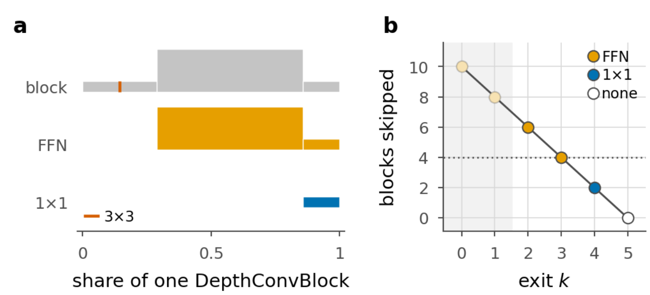

Figure 3. Inside an exit adapter. a, one DepthConvBlock and the two adapters drawn to scale and aligned at their right-hand ends. Bar length is a convolution's share of the block and bar height is the channel width it writes, so each adapter is visibly the tail of the block it stands in for: the FFN pair at 0.71 of a block and the single pointwise at 0.14. The hairline rule inside the block is its only 3×3, and neither adapter contains one, so no adapter adds receptive field and none contributes any tile-boundary penalty. **b**, the blocks each exit skips, coloured by the adapter it wears. Capacity is matched to the gap, the rule switching to the FFN at four skipped blocks (dotted), and the adapter's own cost is charged against the saving, so an exit-2 tile saves six blocks less 0.71. The faded exits lie below the split depth, where no tile can leave. Every length is counted with hooks off the modules themselves by scripts/adapter_cost.py.

3.3 Exit adapters

An early exit hands the shared head a feature the head was not fitted to, and the adapter is the correction. For exits that skip little we use a residual 1×1, Ad(f) = f + Wf with W zero-initialised. For exits that skip four blocks or more we use the pointwise expand/activate/contract pair of the block's own FFN. Both are *pointwise by design*. A 3×3 inside an adapter would add seam damage at the tiles that took an early exit, and those are the tiles least able to afford it. Zero-initialising the last layer makes every adapter exactly the identity at step zero, so the deepest exit is bit-exactly the released decoder before training begins and every shallow exit starts from “the decoder as it is”.

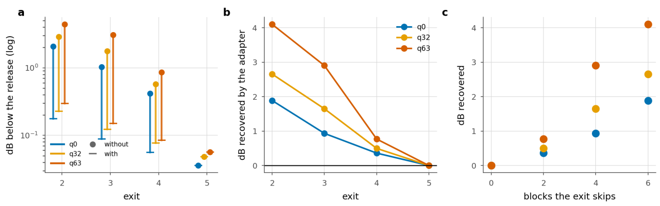

Figure 4. What the adapters are worth. a, each exit with and without its adapter. **b**, the dB the adapter recovers. **c**, the same against the number of blocks the exit skips, which is the design rationale measured.

	q0	q32	q63			
Exit	with	without	with	without	with	without
2	0.177	2.065	0.228	2.884	0.297	4.398
3	0.089	1.023	0.123	1.772	0.151	3.054
4	0.056	0.416	0.077	0.574	0.084	0.853
5 †	0.036	0.036	0.048	0.048	0.056	0.056

Table 2. What the adapters are worth. dB below the release with every tile at that exit, with the trained adapters and with each set back to the identity it was initialised to. † the deepest exit has no adapter by construction and is the control.

Zeroing the adapters measures what they learned, since the identity is exactly what they were initialised to. Without them the shallowest exit costs 4.40 dB at q63, forty-four times the budget, and the adapter buys 4.10 dB of that back. We find the gain grows with rate and with the number of blocks the exit skips, so the design rationale here is a measurement rather than an argument, and the deepest exit moves by exactly zero. We would not describe the adapters as a refinement of the ladder, because without them it has no usable rung.

3.4 Allocation

Given per-tile distortions D(t,k) and costs c_k, the allocation that minimises distortion at a compute budget is the Lagrangian

$$k^*(t) = \arg\min_k \, [D(t,k) + \lambda c_k] \tag{1}$$

with λ found by bisection so the frame lands on the quality budget. Both quantities are available *to the encoder*, which holds the source. That gives configuration **A**. The encoder searches and transmits the map, which entropy-codes to ~89 bits per 1080p frame, or 0.021% of a typical bitrate. It costs the encoder about one extra decode and the decoder nothing.

The decoder cannot evaluate it, because D(t,k) is the error against a source it never sees. What is missing is a variable, so no architecture recovers it and no amount of estimation capacity substitutes for it. Configuration **B** therefore *predicts*. A 144K-parameter head reads the full-frame stem feature, the decoded latent y-hat, the entropy model's scales and the quality index, emits logits z_t, and decides

$$\hat{k}_t = \arg\max_k \, [\log \mathrm{softmax}(z_t)_k - \beta c_k] \tag{2}$$

with β bisected exactly as λ is. Nothing is added to the file. We train the head against a *frozen* decoder with a cost-sensitive cross-entropy to the oracle's choice, each tile weighted by the regret of choosing wrongly. Load balancing is loss-free [16], and we bias it toward the oracle's own exit distribution instead of toward uniform. At a high λ the oracle genuinely does send every tile to one exit, and forcing spread there would force mistakes.

3.5 Training

All exits are decoded every step and the objective is

$$\mathcal{L} = \mathcal{L}_{\mathrm{RD}} + w_a \mathcal{L}_{\mathrm{anchor}} + w_d \mathcal{L}_{\mathrm{distill}} \tag{3}$$

where L_RD is the released rate-distortion loss averaged over exits. The anchor term L_anchor pins the deepest exit to the released decoder, so the reference every saving is quoted against cannot drift away underneath the measurement. The distillation term L_distill supervises the adapters in feature space. We work in feature space and not in pixels because the head is a fixed map from feature to RGB, so matching the deeper feature is the stronger constraint, with 384 dense channels of target instead of 3. Each exit imitates its neighbour, exit k following exit k+1, and not the deepest exit, for the reason given in [19].

We train the adapters *through the tiled decode path they are deployed in*. Training them full frame and tiling only at inference loses 0.14–0.24 dB; training through the deployed path gains 0.51–0.90 dB, so the sign of the effect flips.

4. The price of tiles

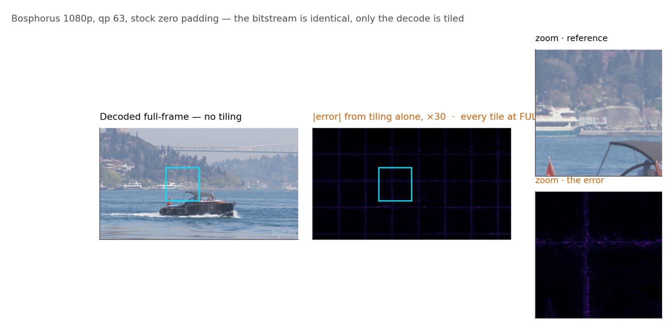

Figure 5. The artefact, before anything is done about it. One frame decoded twice from the *same* bitstream with the same weights, every tile at full depth, stock zero padding, and no early exit anywhere. The only difference is that the right-hand decode was tiled. The error map is the tile lattice and nothing else.

A 3×3 depthwise at feature position (i,j) computes a weighted sum over its neighbours. Decoded full frame, those neighbours exist. Decoded per tile they do not, and the kernel is handed whatever the padding rule invents. Each convolution extends the contaminated region by one ring, so with b per-tile blocks on a tile of side F the fraction of the tile within reach of an invented value is

$$1 - \left(\frac{F-2b}{F}\right)^2$$

At our shipped F=32, b=8 that comes to 0.750. Three quarters of the tile is affected, so what we are looking at is a structured error over most of the tile and not a thin border at its edge.

That fraction tells us which pixels are affected and not how badly, and it turns out to be the wrong predictor of the penalty. We swept the split depth, which sweeps b from 12 to 0 with b=0 as an exact zero-seam control, and measured

$$\text{seam} \propto b^{\alpha}, \qquad \alpha \approx 2 \tag{5}$$

with α = 2.38 at q0, 2.22 at q32 and 1.93 at q63, and r = 0.98–0.995 in log-log. Fitted with one free scale, the area fraction is wrong by 160–264% where the power law is wrong by 12–22%. Fixing the exponent at exactly two costs a factor of two, 31–38%, so the square is the right shape without being quite the right law. The exponent has a reading. The count of contaminated pixels grows like perimeter times depth, and the error accumulated in each of them grows with how many convolutions reached it. Once every pixel has been touched the area law saturates and the penalty does not, which is where the area law fails as a predictor.

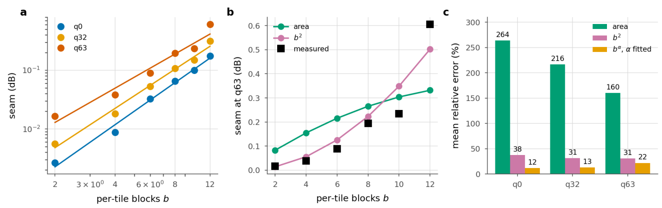

Figure 6. The seam against per-tile depth. Sweeping the split depth sweeps b; b=0 is an exact control and measures 0.0000 dB at all three rates. **a**, the measurement with the fitted power law. **b**, both models against q63 with one free scale each. **c**, mean relative error. The area fraction saturates once every pixel is contaminated; the penalty does not.

4.1 Padding is an estimator

Border padding is an *estimator* of the unseen neighbour, and the seam is its error. The table below measures four of them, with early exit switched off so that tiling is the only difference from a full-frame decode.

Border estimator	q0	q32	q63
Zeros (stock)	0.126	0.287	0.677
Replicate	**0.071**	**0.123**	**0.218**
Linear extrap.	0.198	0.286	0.384
AR(1) per channel	0.061	0.107	0.197

Table 3. Border estimators, dB below the released decoder on the same latent, 256 px tiles, full CTC. Lower is better. The zeroth-order hold beats the first-order extrapolation by a wide margin, and the fitted AR(1) wins on quality while costing +10.7% of decode wall-clock.

A higher-order estimator is worse. Extrapolating the local gradient past a boundary amplifies whatever noise sits on that boundary, and assuming local constancy does not. The gap is wide, 0.384 dB against 0.218 dB at q63. **The best estimator loses on cost.** The per-channel AR(1) fit of [11] reaches 0.197 dB, about 0.02 dB better than replication, and it costs 10.7% of decode wall-clock. Against a 0.1 dB budget and a ~24% saving that trade does not close, so we drop it.

4.2 What actually removes the seam

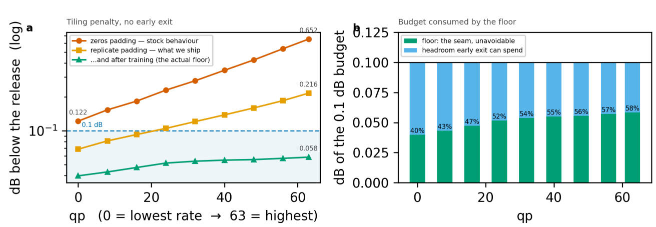

Figure 7. The tiling penalty across the whole rate range, every tile at full depth. The two steps that matter cost nothing, and the largest single factor is training the ladder with the seam present. Panel (b): the floor is charged *inside* the quality budget and consumes a growing share of it.

Tile size is free in arithmetic. A tiled decode's MAC count does not depend on it at all, and the damage scales with the tile perimeter, as the table below shows. What tile size costs is routing granularity, 40 tiles per 1080p frame at 256 px against 160 at 128 px.

Estimator, q63	128 px	256 px	ratio
zeros	1.249	0.677	×0.54
replicate	0.372	0.218	×0.58
arls	0.332	0.197	×0.59

Table 4. Tile size, dB at q63. Doubling the side roughly halves the penalty, as the contamination law predicts, and costs no computation.

The largest factor of all is the easiest one to miss. On the untrained ladder, replicate padding leaves 0.218 dB at q63; after training, the same configuration leaves 0.072 dB. So more than two thirds of what the estimator could not fix is absorbed by weights learning to live with it. That single change removes more of the seam than any module in this paper does.

4.3 A learned repair module, and why we reject it

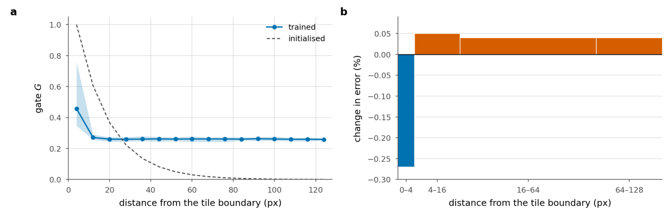

Figure 8. Grid seam repair. A correction gated by position within a tile, applied once to the stitched frame, for 0.95% of the decode. **a**, the gate against distance from the tile boundary, beside the initialisation it started from; shading is the spread over cells at the same distance. The trained gate keeps the ring, up to 0.76 at a tile corner, and then flattens onto a floor of 0.26 instead of switching off. **b**, the change in error measured with the repair on, by distance from the nearest tile boundary; bar width is the share of pixels, so bar area is what a band contributes to the frame. It wins 0.27% on the 6.2% of pixels within 4 px of a boundary and loses 0.04 to 0.05% on the 94% beyond it.

The tile lattice is known exactly at training and at inference, so a repair can be *told* where to look instead of having to infer it

$$\mathrm{Rep}(f) = f + G[\, i \bmod P,\ j \bmod P\,] \cdot \mathrm{PW}(\mathrm{WSiLU}(\mathrm{DW}_{3\times 3}(f))) \tag{6}$$

with G a P×P gate shared over channels and initialised at $\exp(-d/\tau)$. It costs 0.95% of the decode.

It does not earn that. Splitting the per-pixel error by distance from the nearest tile boundary, we find the module gains 0.27% in the 0–4 px band, which is 6.2% of pixels, and loses 0.04–0.05% everywhere else. Give it a *perfect* gate, with zero correction in the interior and the boundary gain unchanged, and the ceiling on what it could earn is ≈0.0008 dB, for 0.95% of the decode. We rejected AR(1) padding at 0.0019 dB per point of decode, and this is worse by an order of magnitude. Tightening the gate cannot rescue it, because there is almost nothing left to win.

4.4 Removing the cause, and why it does not help

Only 0.29% of each block has spatial extent, so the exact fix is affordable. Give the 3×3 its real neighbour from a shared canvas and the cost is +0.032% of the decode, thirty times less than the repair module. The fix is also exact. At uniform depth a coupled tiled decode is bit-identical to a full-frame one, once the comparison is not confounded by the repair module, which runs on the stitched canvas and has no full-frame counterpart. Measured: 1.13e-2 with the repair on, **exactly 0** with it off, against 6.06e-2 for replicate padding.

Coupling also removes most of the floor. Switched on at inference, it drops the floor from 0.036 to 0.003 dB at q0 and from 0.056 to 0.030 at q63, which is 91% and 47% of the tiling penalty.

And it destroys the allocation. At the same 0.1 dB budget the saving falls from 25.9% to 4.2% at q32 and from 19.2% to 0.4% at q63.

The reason is structural, and it is written into the condition on the exactness claim. Coupling is exact when every tile is at the *same* depth, and routing deliberately violates that condition. With replicate padding a tile is entirely independent of its neighbours, so the allocation is free to give adjacent tiles any depths it likes. Coupling makes a tile depend on its neighbours' features, and under routing those neighbours ran a different number of blocks. A shallow tile reading a deep neighbour's activation is a configuration the weights have never seen, and that mismatch costs more than the seam it removed.

The natural reading of the exactness result is that the exact fix should simply be adopted, and that reading is wrong. We report the negative result for that reason, and because the tension between coupling and routing belongs to the combination itself, so any spatially-adaptive decoder will inherit it. One caveat keeps the finding from being final, the same one that applies to the repair module. This model was trained with padding, and a run trained *with* coupling could reverse the result. The burden of proof lies with that run.

5. Experiments

Setup.

We fine-tune the decoder on 512×512 crops from OpenImages with the encoder frozen ($\max|\Delta| = 0$ is asserted every run). One λ is drawn per sample from a log-spaced range covering all 64 quality indices, so a single set of weights covers the whole rate range. We evaluate on the 53 sequences of the common test set (UVG, MCL-JCV and HEVC classes B, C, D and E), one intra frame each.

Measurement protocol.

Two details of the protocol move the numbers enough that we state them here. The per-tile distortion table has to be built on the *deployed* decode path, that is, one tiled decode per exit, rather than by tapping the exits of a single full-frame forward pass. Tapping is the natural implementation and it is correct for training. But with a full-frame reference on the other side of the ratio, the tiling penalty cancels, and the quality reported is that of a decoder nobody ships. On our model the gap is +0.035 dB at the deepest exit and +0.008 dB at the shallowest. It is not a constant offset; it grows with how many blocks ran per tile, so we cannot correct for it after the fact. The second detail is that once the allocation is chosen, we decode that *mixed* map once and report the distortion it actually produces.

Reporting conventions.

Every dB is measured against the *released* decoder's full-frame decode of the *same* latent, and our side is the deployed tiled decode, so the tiling penalty sits inside every number. Savings are fractions of the released decoder's cost. Our own deepest exit costs 1.0095 of it, and using that as the denominator would flatter every result by 0.6–0.8 points.

5.1 Main result

Figure 9. What the saving looks like. The same bitstream decoded by the released decoder and by ours at the 0.1 dB operating point, 30.5% fewer multiply-accumulates. The crop is the tile that gave up the most quality, chosen automatically, so it shows the method's worst case on this frame rather than a flattering one.

Budget	q0	q16	q32	q48	q63	mean
0.10 dB	29.8	25.5	20.7	18.2	15.6	**22.0**
0.30 dB	39.1	39.1	39.1	37.9	35.8	**38.2**
0.50 dB	39.1	39.1	39.1	39.1	39.1	**39.1**

Table 5. Decoder MACs saved (%) against the released decoder, per quality index and quality budget, configuration A. The 0.3 and 0.5 dB rows sit on the architectural ceiling almost everywhere; past that point a looser budget buys nothing.

The table carries the headline numbers. At 0.1 dB the method saves 29.8% at the lowest rate and 15.6% at the highest, and 22.0% on average, at a BD-Rate cost of 0.88%; that is, the compute is worth about the same as a 0.88% increase in bitrate. We do not read the fall with rate as an artefact of the ladder. High-rate reconstructions carry detail the shallow exits cannot reproduce, and the floor rises with rate as well; both effects push the same way.

Decoder	GMAC	% of release	ms
Released DCVC-UF	454	100.0	111
FLEX-UF, all tiles deepest	458	101.0	114
FLEX-UF, 0.1 dB budget	354	78.0	78
FLEX-UF, architectural ceiling	263	58.1	–

Table 6. Decoder complexity at 1080p. Our deepest exit costs slightly more than the release because it still pays the seam-repair module; that 1.0095, not 1.0, is what every saving in this paper is *not* divided by. Wall-clock is the sorted loop of Section 5.9.

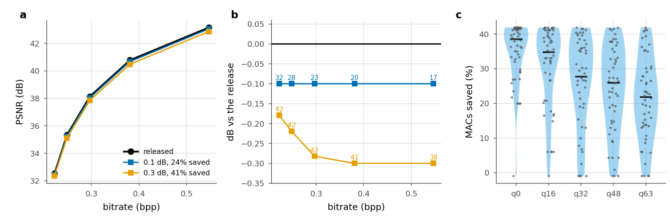

Figure 10. The plane a codec is read on, and the spread behind the mean. a, operating points against the released curve. **b**, the same with the quality axis expanded; labels are compute saved. **c**, per sequence at a matched point near 0.1 dB; one dot per sequence, bar is the median.

Panel c shows the distribution that the headline averages over, and it is wide. At q0 the median sequence saves 38.6%, with an interquartile range of 32.0–41.5, while the worst saves −1.0%. The worst cases are the low-resolution sequences of Section 5.3, where two tiles leave nothing to allocate. Reporting the mean alone would hide both ends.

5.2 Is per-tile adaptivity necessary?

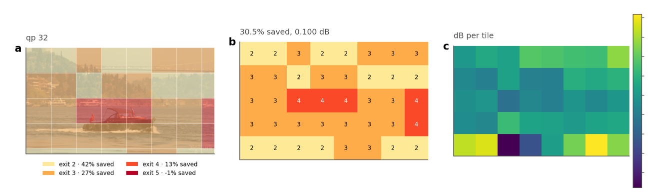

Figure 11. Where the decoder spends. Bosphorus at q32, a 0.1 dB budget. (a) the assignment overlaid on the frame, (b) the exit index per tile, (c) the quality each tile gives up, against its *own* full-depth reference. Water and sky leave at the shallowest rung; the boat and the shoreline run deep.

q	Allocation	ΔPSNR (dB)	MACs saved (%)
0	uniform, exit 2†	0.179	39.1
	uniform, exit 3	0.093	24.2
	uniform, exit 4	0.059	13.0
	uniform, exit 5	0.036	-1.0
	random (matched mix)	0.125	29.9
	rate-ranked (free)	0.107	29.9
	oracle	**0.100**	**29.9**
16	uniform, exit 2†	0.219	39.1

q	Allocation	ΔPSNR (dB)	MACs saved (%)
	uniform, exit 3†	0.118	24.2
	uniform, exit 4	0.075	13.0
	uniform, exit 5	0.045	-1.0
	random (matched mix)	0.133	25.6
	rate-ranked (free)	0.106	25.6
	oracle	**0.100**	**25.6**
32	uniform, exit 2†	0.282	39.1
	uniform, exit 3†	0.147	24.2
	uniform, exit 4	0.090	13.0
	uniform, exit 5	0.054	-1.0
	random (matched mix)	0.141	20.9
	rate-ranked (free)	0.107	20.9
	oracle	**0.100**	**20.9**
48	uniform, exit 2†	0.333	39.1
	uniform, exit 3†	0.176	24.2
	uniform, exit 4†	0.103	13.0
	uniform, exit 5	0.062	-1.0
	random (matched mix)	0.145	18.3
	rate-ranked (free)	0.107	18.3
	oracle	**0.100**	**18.3**
63	uniform, exit 2†	0.396	39.1
	uniform, exit 3†	0.206	24.2
	uniform, exit 4†	0.116	13.0
	uniform, exit 5	0.072	-1.0
	random (matched mix)	0.141	15.6
	rate-ranked (free)	0.110	15.6
	oracle	**0.100**	**15.6**

Table 7. Adaptivity against two controls at the 0.1 dB budget. † marks uniform depths whose distortion exceeds the budget, so they are not admissible allocations at all. “Random” draws each frame's map from the oracle's own exit histogram and shuffles it across tiles, which preserves the average cost and the mix of depths while discarding the content dependence.

The obvious alternative to routing tiles is to run every tile at the same shallower exit and accept the loss. The table puts that option, together with two stronger controls, at matched compute.

The uniform rows answer the question directly, and the answer sharpens with rate. At the lowest rate a static decoder can reach exit 3 inside the budget and save 27.0%, against the oracle's 29.8%. At q48 and q63 the only uniform depth that fits is the deepest one, which *costs* 0.95% instead of saving anything. At those two rates, then, the comparison is between 17% and nothing at all, not between 17% and something smaller. Averaged over rates, the best static allocation saves 9.7% where the adaptive one saves 22.0%.

The two shuffled rows separate effects that are easy to conflate. We keep the oracle's own exit histogram, so the mix of depths and hence the average cost are unchanged, and assign it to tiles at random. Quality collapses. What the allocation buys is knowing *which* tiles can afford to run shallower; running some fraction of them shallower is worth nothing by itself. The rate-ranked row keeps the same histogram and orders it by a signal the decoder already holds, namely the bits the entropy model spent on each tile. This is the cheapest router we can think of. It has no parameters and no training, and the number is available before the trunk starts. It is also the natural analogue of the confidence rules used by early-exit classifiers [3, 20], which likewise read a signal off the network's own output.

It recovers most of the gap. At q63, random costs 0.141 dB and the oracle 0.100 dB at identical compute, while rate-ranking costs 0.110 dB. That is 75% of the oracle's advantage over chance, at 0.68–0.79 agreement with the oracle's map. A learned router therefore has to beat a free baseline that is already three quarters of the way there. We would not have run that comparison without this control, and we suggest that any adaptive-inference paper report it. Given the oracle's histogram, what we measure here is *ranking* and nothing else. Section 5.6 removes that crutch and turns the same signal into a complete routing rule, which matches the trained head across the whole rate range.

5.3 Resolution, and the granularity of a tile

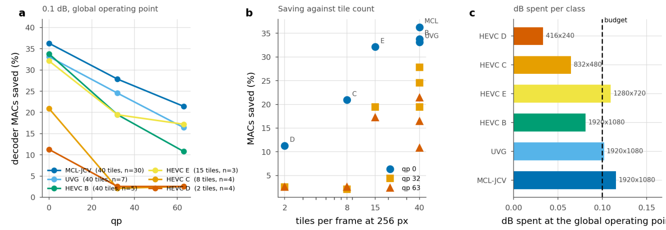

Figure 12. Saving by test class at one global operating point. λ is bisected once so the whole set lands on the budget, exactly as it would be in deployment; each class is then reported at that λ. The ordering follows tile count, not content.

Class	Resolution	Tiles	n	q0	q32	q63
MCL-JCV	1920x1080	40	30	33.7	26.2	20.2
UVG	1920x1080	40	7	30.6	23.1	15.5
HEVC_B	1920x1080	40	5	31.1	18.2	9.7
HEVC_E	1280x720	15	3	29.1	18.4	15.5
HEVC_C	832x480	8	4	18.0	1.7	2.5
HEVC_D	416x240	2	4	8.8	2.5	2.5

Table 8. Saving by test class (%), at a 0.1 dB budget set globally over all 53 sequences rather than per class. “Tiles” is how many 256 px tiles a frame of that resolution contains. The saving tracks tile count much more closely than it tracks content.

Completing the test set exposed a dependence that the 1080p-only subset had hidden. The table groups the saving by class. At 1080p, where a frame is 40 tiles, we save 33.7% (MCL-JCV), 33.1% (UVG) and 33.8% (HEVC B) at the lowest rate, three classes of very different content within three points of each other. At 832×480 a frame is 8 tiles and the saving is 18.0%; at 416×240 it is 2 tiles and 8.8%. At high rate the small resolutions collapse to 2.5% against 20.2% for 1080p.

The natural explanation is granularity. With two tiles per frame there is almost no allocation left to make, and the Lagrangian degenerates towards a uniform choice. That suggests a fix. Choose the tile size relative to the frame, since the MAC count does not depend on tile size at all and only the seam does.

We tested that fix and it mostly does not work. We compare the 256 px configuration against a 128 px one at q0, which multiplies the tile count by 3.4. MCL-JCV (1080p) goes from 36.3 to 34.3, HEVC B (1080p) from 33.8 to 32.1, HEVC C (832×480) from 20.9 to 17.6, and HEVC D (416×240) from 11.3 to **13.7**.

Smaller tiles help only at the smallest resolution, where the count goes from two to eight. Everywhere else they cost between 1.7 and 3.3 points, including at 832×480, where the count rises from 8 to 28 and the saving still falls. Beyond a modest number of tiles, the extra seam outweighs the extra granularity. We report the comparison as indicative, since the two configurations come from different training runs and tile size is therefore confounded with training. But the direction is consistent, and it is enough for us to say that a resolution-adaptive tile size is not the easy win the granularity argument suggests.

5.4 The working range of a quality budget

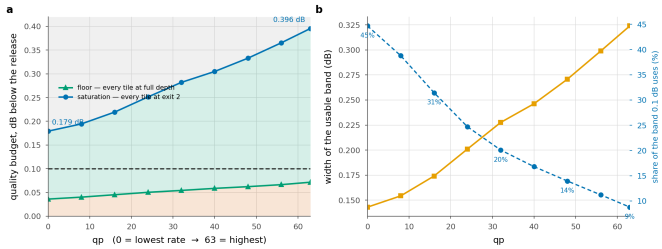

Figure 13. Three regions, and only the middle one is a design choice. Below the floor no allocation meets the budget; above saturation every tile is already on the cheapest rung. The working 0.1 dB budget uses 45% of the window at the lowest rate and 9% at the highest.

q	0	16	32	48	63
Floor D_min	0.036	0.045	0.054	0.062	0.072
Saturation D_sat	0.179	0.219	0.282	0.333	0.396
Usable band	0.143	0.174	0.228	0.271	0.324
0.1 dB uses	45%	31%	20%	14%	9%

Table 9. Floor and saturation, dB below the released decoder. The floor is what tiling costs with no early exit at all; saturation is where every tile reaches exit j and the ceiling is attained.

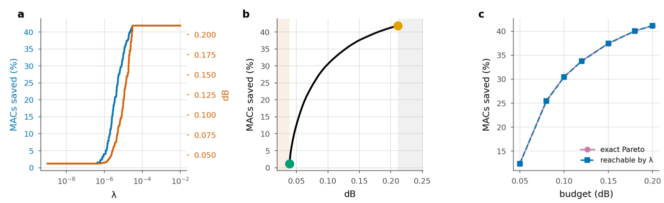

Figure 14. The structure of the allocation, on one frame. **a**, compute and distortion are monotone in the multiplier. **b**, the frontier between the floor (green) and saturation (orange); shaded regions are infeasible and wasted. **c**, what the Lagrangian reaches against the exact Pareto set, enumerated by dynamic programming; the two curves are indistinguishable.

The construction has enough structure to state as propositions, and we check each one numerically instead of asserting it (scripts/verify_theory.py, seven of seven). The allocation decouples per tile; compute is non-increasing and distortion non-decreasing in λ; the sweep traces the lower convex hull and therefore cannot reach an interior point of the achievable set; λ=0 reaches the least distortion the ladder can produce; beyond a finite λ, computable in closed form, the allocation is the constant map to exit j at cost exactly c_j; and the ceiling is a function of the split depth alone.

The third of those has a practical edge. The sweep reaches only hull vertices, so a budget that falls between two of them cannot be met exactly. We measured what that costs by enumerating the *exact* Pareto set with dynamic programming over tiles, which is feasible because the cost alphabet has four symbols. The sweep reaches 95 allocations against a Pareto set of 635, and the loss from convexity is at most 0.05 saving points. For this problem the standard construction is essentially optimal.

It matters *why* that loss is small, since the reason is structural and points at what would make it smaller. The reachable costs form a lattice, and its spacing is set by how much the mean cost moves when the multiplier crosses a switching point. If m tiles each move one rung there, the spacing is at most $m \cdot \max(c_{k+1} - c_k)/T$ saving points. The convexity loss cannot exceed the largest spacing, because a budget between two levels is served by the lower one. Here m=2, since tiles with identical rows switch together, and the bound is 0.745 points; the measured spacing sits exactly on it. Nothing in the expression depends on content or rate, and the bound falls as 1/T: halving the tile side puts four times as many tiles on the lattice, each carrying a quarter of the step. Fine granularity is what makes the Lagrangian relaxation lossless in practice, and no property of the images is doing that work.

Two numbers bound what any budget can do. The **floor** is the distortion of a tiled decode with every tile at full depth, which is pure tiling penalty: 0.036 dB at q0, rising to 0.072 dB at q63. A budget below it admits no allocation. The **saturation** point is the distortion when every tile takes exit j; any budget at or above it reaches the ceiling, and a larger one reaches nothing more.

One consequence of that is worth stating plainly. At q0 the ceiling is reached at 0.179 dB, so the architectural bound behaves as an operating point rather than as an asymptote. Past it, the limiting factor is the *ladder* and not the budget, which is an argument for a finer ladder before a looser budget. The same two numbers pick out a narrow regime: budgets in [0.179, 0.194) dB saturate the lowest rate and nothing else, a window 15 millibels wide.

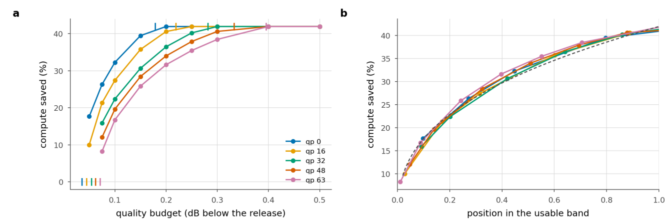

Figure 15. The band is the rate dependence. a Saving against the budget, with each rate's floor and saturation point ticked. **b** The same with the budget axis rescaled onto each rate's own band. Five curves become one; dashed is the one-parameter power law fitted to all of them.

And the window is almost all of the rate dependence. At a matched decibel the five rates are 15.5 points apart. Rescale the budget axis onto each rate's own band, with the floor at 0 and saturation at 1, and they collapse onto a single curve, 1.7 points apart on average and 2.2 at worst. The answer to "how much does a 0.1 dB budget buy at this rate" is therefore, to within a couple of points, "where does 0.1 dB sit in this rate's band". The floor and the saturation point are both in closed form and both cheap to measure. What they leave over is small enough that a deployment could calibrate the two ends and read the rest off one curve. That curve is a one-parameter power law, saving $\approx C \cdot u^{0.38}$, with C the architectural ceiling and u the position in the band. We fit it in log space over all five rates and get R^2 = 0.989, worst residual 2.0 points.

It is a property of the ladder, not of this checkpoint. We repeated the measurement on a different training run, BEST, a separate recipe taken at a different epoch, and the same thing happens. The rates are 16.9 points apart at a matched decibel and 1.6 after rescaling, and the collapsed curve is again a power law, R^2 = 0.984. The *exponent* does not carry over: 0.33 there against 0.38 here, so it describes a trained decoder and not the architecture. What replicates is the collapse. Within a checkpoint, the rate dependence of the trade-off is the rate dependence of the band.

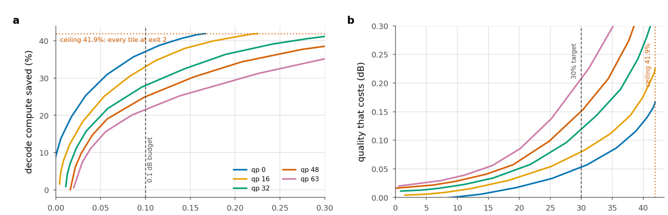

Figure 16. The trade-off, whole. a What a budget buys, per rate; the ceiling is 39.1% and q0 reaches it at 0.179 dB. **b** The same relation inverted, so it reads as what a saving target costs. The three budgets reported elsewhere are three points on this curve.

5.5 Signalled versus predicted allocation

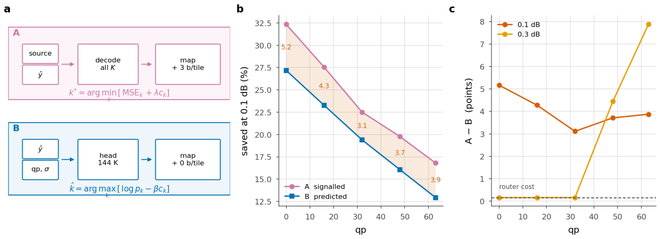

Figure 17. Who decides. (a) A holds the source, so it can decode all K exits per tile and pick the true minimiser; it signals the map at ~89 bits/frame. B never sees the source and signals nothing; its 144 K-parameter head reads y-hat, the entropy scales and qp. **(b)** Saving at 0.1 dB; shading is what a bit-exact bitstream costs. **(c)** That cost is smallest at q32, where the tilt needed to move the router off its training multiplier is essentially zero, and grows in both directions. At 0.3 dB it collapses to the router's own 0.163% wherever the budget saturates the ladder, and widens where it does not.

	0.1 dB			0.3 dB		
q	A	B	gap	A	B	gap
0	29.8	24.4	**5.4**	39.1	39.0	**0.2**
16	25.5	20.9	**4.5**	39.1	39.0	**0.2**
32	20.7	17.4	**3.3**	39.1	39.0	**0.2**
48	18.2	14.1	**4.1**	37.9	33.4	**4.5**
63	15.6	11.1	**4.5**	35.8	27.7	**8.0**

Table 10. Signalled against predicted at two budgets, same checkpoint and test set, with one router trained against the deployed oracle at a single λ.

Configuration A is exact by construction and costs bits; configuration B is approximate and costs none. The table above compares them.

Not signalling costs 3.3–5.4 points, roughly flat across rate, for zero added bits and a byte-identical file. The gap is smallest at q32 and widens toward both ends of the rate range.

It tracks |β|, the tilt the bisection has to apply to move a router trained at one λ onto another operating point. At q32 the required tilt is β=10, essentially none, since that is where the training λ lands, and there the gap is at its minimum. At q0 the tilt is 146 and the gap is 5.4. A large tilt in either direction lets the cost term dominate the logits, which discards the content ranking the router learned. The remedy is a router per operating point, and a deployment would have one anyway: a single set of weights covers all rates, and a 144 K head per rate is 0.3% of the model each.

Loosening the budget closes the gap only where the budget saturates the ladder. At 0.3 dB the gap is 0.2 points at the three lowest rates. That is exactly the router's own 0.163% of decode, so its *prediction* is free there; both configurations send every tile to the cheapest rung, and nothing is left to predict wrongly. At the two highest rates 0.3 dB does not saturate, and the gap *widens* to 8.0 points. A looser budget gives the allocation more room to be wrong in as well as more room to be right.

For a system that trains once, the single-router number is the honest one, so that is what we report. It understates what B can do.

Retraining with a working mask raises agreement and does not simply raise the saving. The head above was trained against a suppression term sitting above its own logits (below). We retrained it with the mask fixed, same recipe and same λ. Held-out agreement rises from 0.718 to 0.808, and the deployed saving moves by +3.1 points at q0 and −3.4 at q63. We find the retrained head ahead where the required tilt is small and behind where it is large, which is the same axis the gap runs along. That is the second time in this paper that agreement with the oracle has moved opposite to the quantity that matters. We keep the original head for every other measurement so the comparisons stay on one system, and we read the retrain as evidence that the mask cost the head real capacity, and that agreement is not the objective.

An earlier version of this measurement put the gap at 1.7 points at q0, rising to 13.3 at q63. The exit mask was at fault. It suppresses exits below the split depth by assigning -1e4, the head's own logits had drifted to that scale, and the suppressed entries were therefore the largest in every row. A large share of every tile went to the cheapest rung for a reason unrelated to its content. The mask is now negative infinity. Fixing it costs 3.5 points at q0, where the accident happened to agree with the oracle, and buys 9.4 at q63, where it did not.

5.6 A router with no parameters

Before a 144 K head is worth its 0.163% of the decode, it has to beat what the decoder already knows. The entropy model has produced one number per tile before the trunk runs, and at no cost: how many bits that tile's latents took.

We turn it into a routing rule with no learned parameters. We model the per-tile distortion as rank-1 in the log domain

$$\log D(t,k) \approx \alpha \log b(t) + c + \log \varphi_k \tag{7}$$

where b is the tile's bit count normalised by the frame mean and ϕ is a K-vector saying what each exit costs on an average tile. We fit (α, c, ϕ) by least squares, leave-one-sequence-out, so that no sequence contributes to the profile that routes it. The same Lagrangian the oracle runs is then run on the surrogate. Nothing is signalled and nothing is trained; the arithmetic is one scalar per tile.

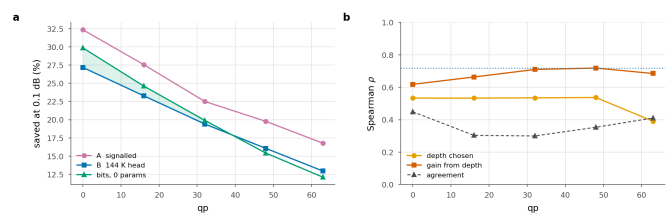

Figure 18. The free baseline. a Saving at 0.1 dB; shaded where the parameter-free rule beats the trained head. **b** What a tile's bit count is correlated with, against how often the rule agrees with the oracle outright; dotted is the head's own held-out agreement.

q	rate-rank	B router	A signalled	ρ depth	ρ spread
0	**27.8**	24.4	29.9	+0.54	+0.61
16	**22.7**	20.9	25.6	+0.52	+0.66
32	**18.2**	17.4	20.9	+0.53	+0.71
48	**14.7**	14.1	18.3	+0.53	+0.71
63	**12.0**	11.1	15.6	+0.38	+0.68

Table 11. The free baseline, 0.1 dB, same test set. Bold where the parameter-free rule beats the trained head. ρ are Spearman correlations between a tile's bit count and, respectively, the depth the oracle assigns it and the distortion it stands to gain from that depth.

It beats the trained head at every rate. The free rule is ahead of the 144 K router at all 5 measured rates, by margins that run from half a point at q48 to 3.4 points at q0. A head trained on this decoder against this oracle therefore returns nothing over a rule with no parameters at all, and it carries 0.163% of the decode that the free rule does not.

Why it works is not that it agrees with the oracle. It agrees on 0.28–0.46 of tiles, well below the router's 0.718, and still saves more at every rate. Agreement weighs a disagreement on a tile where two exits are within a hair of each other exactly as heavily as one where the choice is most of the frame's error, and most tiles are the former. The ordering is what the rule gets right. Bits correlate with the *spread* across the ladder, that is, with how much a tile stands to gain from depth, at ρ_spread = 0.61–0.71 at every rate.

At a looser budget it stops being a baseline and becomes the answer. At 0.3 dB the free rule matches the oracle exactly at the three lowest rates, where both reach the same architectural ceiling, comes within 0.2 points of it at q48, and gives up 34.3% against the oracle's 35.8% at q63. That is ahead of the trained head at every rate, by up to 6.6 points. The head is charged for itself and the free rule is not. At a loose budget the head's ordering also has less left to contribute, because once the budget saturates the ladder there is little ordering left to get right, and the free rule gets it. At 0.5 dB it reaches the ceiling at every rate and its assignment is *identical* to the oracle's on every tile.

What it cannot do is see anything beyond that ordering. A rank-1 model in the level assigns every tile the same relative profile over exits, so b only decides where on the ladder a tile falls, never the shape of its trade-off. That is the ceiling this baseline sits at, and it is the part a learned head should be earning its parameters on. Ours does not earn them at any rate we measured.

q	A signalled	B router	bits, 0 params
0	34.0	31.6	30.9
16	27.7	25.1	**27.1**
32	22.3	16.5	**22.4**
48	19.8	13.3	**19.1**
63	17.2	9.0	**16.2**

Table 12. The same comparison on a second training run (BEST), 0.1 dB, same test set. Bold where the parameter-free rule beats that run's own trained head.

It replicates on a second training run. BEST is a separate recipe at a different epoch, with its own router trained the same way. There the free rule beats the trained head at 4 of 5 rates, by up to 7.2 points, and comes within 3.1 points of the *oracle* everywhere. The margins there are wider than here, and the one rate it concedes is the only rate on either checkpoint where the head finishes ahead, by under a point. We do not read that as showing a learned head cannot be worth its parameters, only that neither of our training runs produced one that is. The free rule stays close to the oracle on both.

Per-block bit allocation is a standard quantity in learned compression, where it is something to *choose*; block-level rate control sets it so that complex regions get more bits [42]. We read the same number in the other direction, after the fact and at the decoder, as a statement about how hard a region was. It costs nothing because someone else has already paid for it.

q	bits	0.1	0.25	0.5	0.75	0.9	head
0	**29.8**	27.9	27.9	27.9	28.0	27.9	28.0
16	**24.1**	23.3	23.0	23.0	22.8	22.7	22.8
32	**19.6**	19.5	19.4	19.3	19.3	19.3	19.3
48	14.9	16.9	**17.0**	16.8	16.7	16.7	16.6
63	12.4	**13.2**	12.4	11.7	11.6	11.4	11.4

Table 13. Blending the two decoder-side signals at 0.1 dB, both normalised to unit mean, weight w from the bit rule to the head's ordering. Bold is the best per rate.

Are the two signals complementary? Barely. We normalise both surrogates to unit mean and blend them with one weight w, where w=0 is the bit rule and w=1 the head's ordering. The best blend is w=0 at the three lowest rates and a small head weight at the two highest, worth +2.1 points at q48 and +0.9 at q63. The head reads the entropy model's scales, so it already has most of what the bit count carries; what it adds is confined to q48 and q63.

A learned component should be measured against the free alternative, and it rarely is. In adaptive inference the usual controls are a uniform allocation and a random one. Both are much weaker than a decoder-side signal that is already lying around.

5.7 Signalling only what the router gets wrong

A and B are the two ends of one scale. The encoder can run the decoder's router, which reads only decoded data, so the encoder knows tile by tile where the prediction will be wrong. Nothing obliges it to correct every one of them.

Configuration C signals a fraction ρ of the tiles and leaves the rest to the router. We choose the overrides by Lagrangian regret, which is exactly what the objective loses on that tile by staying silent. We hold β at B's value and bisect λ over the overrides to land back on the budget. The map costs an entropy-coded mask, N·H(ρ) bits with H the binary entropy, plus 3 per override.

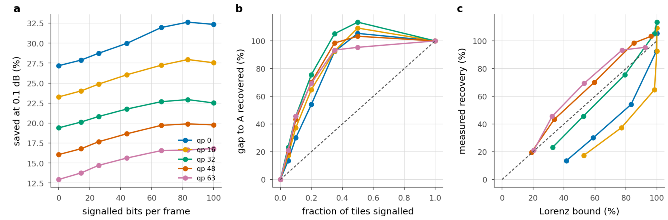

Figure 19. Partial signalling. a Saving against the bits spent on the map; the left end of each line is B and the right end is A. **b** The same, normalised: what fraction of the A–B gap a given fraction of the map recovers; dashed is proportional recovery, and above 100% a partial map is beating a complete one. **c** The same against the fixed-λ Lorenz prediction; dashed is equality, and points above it are allocations a single multiplier cannot reach.

q	B	5%	10%	20%	50%	A
bits/frame	0	14	26	44	83	100
0	24.4	25.1	26.0	27.3	30.1	29.9
16	20.9	21.7	22.5	23.7	25.7	25.6
32	17.4	18.1	18.8	19.8	21.1	20.9
48	14.1	14.9	15.8	16.8	18.3	18.3
63	11.1	11.9	12.9	13.9	15.3	15.6

Table 14. Configuration C at 0.1 dB. Columns are the fraction of tiles the encoder overrides; the two end columns reproduce B and A to within 0.1 points, which is the check that the interpolation is real.

The two ends check out. At ρ=0 the measurement reproduces configuration B to the second decimal at every rate, and at ρ=1 it reproduces A. Neither end is imposed; both fall out of the same code path run against independently measured files, so the columns between them are measuring something real.

Most of the gap is cheap. Overriding a tenth of the tiles for 26 bits per frame recovers 29–39% of the gap, and a fifth recovers 53–62%. Recovery is concave everywhere, so the first bits spent are the most useful ones.

And a partial map can beat a complete one. At 4 of 5 rates, signalling half the tiles for 83 bits saves *more* than signalling all of them, by up to 0.21 points, at the same distortion. Both land within the bisection's 5e-4 dB tolerance of the budget, and the local exchange rate is about 48 saving points per decibel, so the tolerance is worth 0.02 points against a margin ten to twenty times that. The margin is therefore real, and it does not come from a better predictor. The hybrid has *two* multipliers where A has one: the oracle's λ governs the overridden tiles and the router's fixed β the rest. A two-multiplier allocation reaches points on the distortion–compute plane that a single Lagrangian sweep cannot, for the same reason the sweep misses interior Pareto points at all. Configuration A is optimal among allocations reachable by one multiplier, and that is a smaller set than the word suggests.

Why the recovery is concave, and what bounds it. At a fixed λ the objective is separable, so overriding a set removes *exactly* the sum of that set's regrets. The recovery of the *objective* is then the Lorenz curve of the per-tile regret distribution, and its curvature is that distribution's Gini coefficient, 0.49–0.73 across rates. What the table reports is saving at fixed distortion, not the objective, and returning to the budget means re-bisecting λ. That re-bisection is also what lets the two-multiplier allocations above escape the bound.

At the loose budget it matters where the gap is. At 0.3 dB the three lowest rates are saturated, and there is nothing for a partial map to buy. At $q48$ and $q63$, where the gap is 8.0 points, half the map recovers 68–91% of it. The allocation follows the same rule here as everywhere else, which is to spend the bits where the ladder still has somewhere to go.

A better predictor concentrates its regret, which is what the Gini was for. We rebuild C on the retrained head of Section 5.5. The Gini of the per-tile regret rises at 4 of 5 rates, to 0.68–0.91. Its mistakes are rarer and larger. The same table shows the consequence. C converges on A faster from a better base and has less left to recover, and the half-beats-all effect disappears at q0, because a base close to A leaves little for the extra multiplier to exploit. The Lorenz reading is therefore worth having as a diagnostic and not as a decoration; it reports what *kind* of predictor one has as well as how good it is.

Which predictor should C be built on? Either will do, and the answer follows the same split as Section 5.6. Running the identical override rule over the parameter-free surrogate instead of the head is worth up to 3.0 points at the lowest rate and costs up to 0.2 at the highest. Both converge on A at ρ=1 to the second decimal at every rate, which is the third independent check that the two code paths agree. The best single operating point we measured is the free predictor with half the map signalled, at 30.5% at q0. That is 0.61 points *above* full signalling, with no learned component anywhere in the decoder.

Configuration C is what we would ship where a small map is tolerable and a large one is not. It also reframes the A–B gap. What the gap measures is the price of *silence*, charged tile by tile, rather than the price of prediction.

5.8 Does the map have to be recomputed?

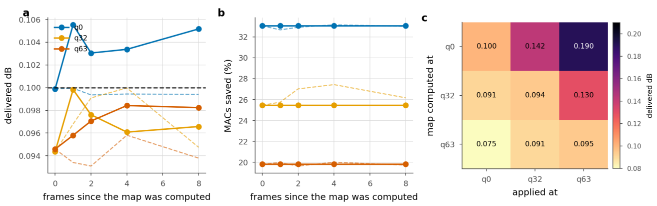

Figure 20. Reusing an exit map. Solid is the transferred map, dashed the one recomputed in place. **a**, **b**: reuse across frames of the same sequence. **c**: reuse across quality index; the entry is the delivered distortion against a 0.1 dB budget.

Configuration A costs the encoder about one extra decode per frame. How much that matters depends on how often the map has to be recomputed, and we have not seen that measured anywhere.

Across frames. Reuse is close to free. We take the map found on frame 0 and apply it eight frames later; at q0 the delivered distortion rises by 0.005 dB, five percent of the budget, and the saving does not move at all: 33.05% transferred against 32.66–33.15% recomputed. The allocation is a property of where the content is hard, and that moves slowly. A codec would search once per group of pictures instead of once per frame, and divide the encoder cost by the group length.

Across rate. Here the map does have to be recomputed, and the direction of the reuse decides how badly. Found at q0 and applied at q63, it delivers 0.190 dB against a 0.1 dB budget, claiming the low-rate saving of 33.05% while spending nearly twice the quality it is allowed. The reverse is safe but wasteful: the q63 map applied at q0 delivers 0.075 dB and only 19.8% where 33.1% was available. Shallow exits are cheap in quality at low rate and expensive at high rate, so a map is calibrated to the rate it was found at, and reusing one upward breaks the quality guarantee without anything in the decode reporting that it has. An encoder should search once per rate and reuse that search across frames.

5.9 Complexity and wall-clock

q	Released	Routed	Saved (%)	
	(ms)	sorted (ms)	measured	MACs
0	111	78	**29.1**	35.3
32	111	86	**22.4**	28.0
63	111	94	**15.6**	20.1

Table 15. Wall-clock, 1080p, median of 40 interleaved iterations on one A100-class GPU, at the 0.1 dB operating point. “MACs” is what the arithmetic predicts; “measured” is the sorted per-tile loop.

A saving in multiply-accumulates is not a saving in time. Tiling costs 3.6% before anything exits early, measured as the deepest-exit tiled decode against the released full-frame one at the same arithmetic and the same weights. On top of that, the routed decode realises 27.9% at q0 where its operations predict 35.3%. Four fifths of the predicted saving arrives. The missing fifth is the tiling overhead together with the bookkeeping of a shrinking active set, and a MAC count sees neither. The shortfall scales with the saving instead of sitting at a fixed offset: at q63 we measure 15.6% realised against 20.1% predicted.

Sorting the tiles once by exit depth recovers part of it. The obvious implementation of a shrinking active set keeps a boolean mask, gathers the survivors and scatters the finished at every group boundary, and each of those boundaries forces a device-to-host synchronisation. Order the tiles by descending exit depth and “still active at group g” becomes a contiguous prefix: each group is then a slice instead of a gather, and one cumulative count supplies every boundary, so no per-group mask is built at all. The finished tiles come back through the inverse permutation in a single scatter. The arithmetic is unchanged and the output is bit-identical on GPU. At q0 the saving goes from 27.9% to 29.1%, a gain of 1.2 points, or a fifth of the shortfall, for a change that touches no arithmetic.

The same gap appears in our own accounting. Every configuration B number in this paper charges the router at its share of the decoder's multiply-accumulates, 0.163%. We also timed the head directly, on the padded 2048×1280 frame the decoder actually sees, interleaved against a deepest-exit decode so that a co-tenant's load falls on both equally. It costs 0.50%, which is 3.0× what its arithmetic predicts, and for the same reason as everything else in this section: a 144 K-parameter head is launch overhead, and a MAC count cannot see a launch. Charged at measured time instead of at operations, every B number in this paper would fall by a further 0.33 points. We leave them charged at MACs because that is the convention the rest of the literature reports in, and we record the correction here so that it does not sit unstated.

A paper that quotes only operations would report 35.3% where the same code, run as written, delivers 29.1%. That is why we give the shortfall as well. The error runs one way: operations are an *optimistic* bound on this method, and the optimism grows with how much of the frame exits early.

5.10 The right ladder depends on the budget

Ladder	Config	Ceiling	0.1 dB	0.3 dB	0.5 dB
RECIPE512	K6 j2 256px	41.9	22.0	38.2	39.1
BEST	K6 j2 256px	41.9	**24.2**	38.5	41.3
BEST128	K6 j2 128px	41.9	19.3	36.6	40.6
FINE12	K12 j4 128px	50.3	19.0*	**42.0**	**48.6**

Table 16. Ladder configurations, mean saving (%) over the five rates, same test set and protocol. “Ceiling” is the architectural ceiling. * one rate is infeasible at that budget, because the ladder's floor exceeds it, so the mean is over the remaining four.

Section 5.4 argued that once a budget saturates a ladder, the only way to spend more is a rung that does not exist. The table measures that. A finer ladder (K=12, j=4) has a ceiling of 50.3% against 39.1%, and the two orderings cross somewhere between 0.1 and 0.3 dB. At 0.1 dB the coarse ladder wins, and the fine one cannot even reach the budget at the highest rate. Splitting later (j=4 against j=2) puts twice as many blocks in the per-tile section, which by the contamination law raises the floor. At 0.5 dB the fine ladder wins by 6.7 points, 48.6% against 39.1%, because the coarse one has been pinned at its ceiling since 0.3 dB and has nothing left to spend.

So the ladder is a choice made at the operating point, not a hyperparameter tuned once and fixed. The floor-saturation window of Section 5.4 tells you which side of the crossover a given budget sits on.

6. Limitations

The seam is reduced, and the exact remedy is not usable as it stands. Canvas coupling removes 47–91% of the floor and is bit-exact at uniform depth (Section 4.4). It also collapses the routed saving, because routing puts neighbouring tiles at different depths. We do not know whether a decoder trained with coupling can recover both at once, and that is the experiment we would run next. Until someone runs it, the floor is a cost this method pays.

Intra frames only. This is the image path of a video codec. Extending the ladder to inter frames raises a question we do not answer here, because an exit map propagates through the reference chain and a shallow tile in one frame is a worse reference for the next one.

Training is not converged. At every checkpoint we have measured more than once, the saving was still going up. We therefore read the numbers reported here as a lower bound on what a converged run would give.

A single decoder. All results are on DCVC-UF's intra decoder. Nothing in the method looks specific to it, since the ladder needs only a residual trunk with a shared head, but we have not measured a second decoder to check.

7. Conclusion

The intra decoder of DCVC-UF can be run at a depth chosen per tile from what that tile contains. At a 0.1 dB budget this removes 29.8% to 15.6% of its multiply-accumulates for a BD-Rate cost of 0.88%, with no change to the encoder and none to the coded payload.

Three lessons we would carry to any spatially adaptive decoder, and none of them is about early exit.

Tiling is the dominant cost and it is governed by depth. All of that cost traces back to a single 3×3 that is 0.29% of the arithmetic, and the penalty grows as the square of how many such convolutions run per tile. The corrupted-area fraction usually quoted alongside it predicts that penalty badly.

The exact remedy is in tension with the thing it enables. Giving each convolution its real neighbour is bit-identical at uniform depth, and under routing it collapses the allocation, because routing is the deliberate violation of the condition that makes it exact. We expect any method that combines spatial adaptivity with tiled inference to walk into this.

A quality budget is only a control variable inside a measurable window. Below the floor the budget admits nothing at all, and above saturation more of it buys nothing. A saving quoted without saying where in that window it sits has left out the part of the result a reader most needs.

We would attach three cautions to the measurements themselves. A saving in operations is an optimistic bound on a saving in time, and the optimism scales with the saving. A learned router should be compared against a free one. Routing on the bits already spent per tile needs no parameters and no training, it adds nothing to the stream, and it beats our trained head at every rate we measured, while agreeing with the oracle on fewer tiles than the head does. We read that as a warning about the metric quite as much as about the head. Finally, a timing harness will report numbers whether or not it is timing the right device. Ours timed the wrong one for months, and what gave it away was a decoder that appeared to slow down with the quality index.

References

[1] S. Wang et al. Adaptive patch exiting for scalable single image super-resolution. ECCV, 2022.

[2] J. Ballé et al. Variational image compression with a scale hyperprior. ICLR, 2018.

[3] G. Huang et al. Multi-scale dense networks for resource efficient image classification. ICLR, 2018.

[4] G. Bjøntegaard. Calculation of average PSNR differences between RD curves. VCEG-M33, 2001.

[5] B. Bross et al. Overview of the Versatile Video Coding standard. IEEE TCSVT, 2021.

[6] Z. Cheng et al. Learned image compression with discretized Gaussian mixture likelihoods. CVPR, 2020.

[7] W. Fedus et al. Switch Transformers. JMLR, 2022.

[8] G. Huang et al. Glance and Focus networks for dynamic visual recognition. IEEE TPAMI, 2023.

[9] G. J. Sullivan et al. Overview of the High Efficiency Video Coding standard. IEEE TCSVT, 2012.

[10] E. Jang et al. Categorical reparameterization with Gumbel-softmax. ICLR, 2017.

[11] T. Kaseva et al. Per-channel autoregressive linear prediction padding in tiled CNN processing of 2D spatial data. arXiv:2502.12300, 2025.

[12] X. Kong et al. ClassSR: a general framework to accelerate super-resolution networks by data characteristic. CVPR, 2021.

[13] J. Li, B. Li, Y. Lu. Neural video compression with feature modulation. CVPR, 2024.

[14] J. Li, B. Li, Y. Lu. Uniformly accelerated neural video codec with feature refinement. ACM MM, 2024.

[15] Y. Kaya et al. Shallow-deep networks: understanding and mitigating network overthinking. ICML, 2019.

[16] L. Wang et al. Auxiliary-loss-free load balancing strategy for mixture-of-experts. arXiv:2408.15664, 2024.

[17] M. Phuong, C. H. Lampert. Distillation-based training for multi-exit architectures. ICCV, 2019.

[18] S. Scardapane et al. Why should we add early exits to neural networks? Cognitive Computation, 2020.

[19] W. Sun et al. Multi-exit self-distillation with appropriate teachers. FITEE, 2024.

[20] S. Teerapittayanon et al. BranchyNet: fast inference via early exiting from deep neural networks. ICPR, 2016.

[21] B. Bross et al. Versatile Video Coding. IEEE TCSVT, 2021.

[22] F. Yang et al. Slimmable compressive autoencoders for practical neural image compression. CVPR, 2021.

[23] M. A. Y■lmaz et al. Slimmable video codec. CVPRW, 2022.

[24] A. Kuznetsova et al. The Open Images Dataset V4. IJCV, 2020.

[25] A. Mercat et al. UVG dataset: 50/120fps 4K sequences for video codec analysis. ACM MMSys, 2020.

[26] H. Wang et al. MCL-JCV: a JND-based H.264/AVC video quality assessment dataset. ICIP, 2016.

[27] T. Blard et al. Spatial competition for low-complexity learned image compression. arXiv:2605.13243, 2026.

[28] Y. Shoham, A. Gersho. Efficient bit allocation for an arbitrary set of quantizers. IEEE TASSP, 1988.

[29] A. Ortega, K. Ramchandran. Rate-distortion methods for image and video compression. IEEE SPM, 1998.

[30] Adaptive test-time compute allocation for reasoning LLMs via constrained policy optimization. arXiv:2604.14853, 2026.

[31] H. A. Alhaija et al. Local padding in patch-based GANs for seamless infinite-sized texture synthesis. arXiv:2309.02340, 2023.

[32] C. Innamorati et al. Overlap training to mitigate inconsistencies caused by image tiling in CNNs. arXiv:1812.02203, 2018.

[33] Z. Jia et al. Towards practical real-time neural video compression. CVPR, 2025.

[34] B. A. Hasan et al. Adaptive inference: theoretical limits and unexplored opportunities. arXiv:2402.04359, 2024.

[35] Y. Gao et al. Exploring the rate-distortion-complexity optimization in neural image compression. CVIU, 2024.

[36] Y.-H. Ho et al. On the rate-distortion-complexity trade-offs of neural video coding. arXiv:2410.03898, 2024.

[37] C. Zhang, W. Gao. Learned rate control for frame-level adaptive neural video compression via dynamic neural network. arXiv:2508.20709, 2025.

[38] Y. Geifman, R. El-Yaniv. Selective classification for deep neural networks. NeurIPS, 2017.

[39] D. Madras et al. Predict responsibly: improving fairness and accuracy by learning to defer. NeurIPS, 2018.

[40] H. Mozannar, D. Sontag. Consistent estimators for learning to defer to an expert. ICML, 2020.

[41] G. DeSalvo et al. Budgeted multiple-expert deferral. arXiv:2510.26706, 2025.

[42] Accelerating block-level rate control for learned image compression. arXiv:2409.01009, 2024.